

AlertSphere — Project Guide

A “professor style” deep-dive explaining how your Spring Boot backend works: APIs, security (Spring Security + JWT), business layer, MongoDB/Redis/Kafka/email, and common interview questions.

Project type	Spring Boot REST API backend
Main domain	Weather alerts (rule engine + scheduled checks + notifications)
Key integrations	MongoDB, Redis, Kafka, SMTP email, OpenWeather API
Auth model	JWT (stateless) + Spring Security role-based access

Table of Contents

1. [One-page overview](#)
2. [Architecture & request lifecycle](#)
3. [API endpoints & how to test](#)
4. [Spring Security + JWT \(deep explanation\)](#)
5. [Business layer \(services\) breakdown](#)
6. [Data layer \(MongoDB\) breakdown](#)
7. [Redis caching + cooldown locks](#)
8. [Kafka event pipeline + reliability](#)
9. [Email delivery \(SMTP\)](#)
10. [Configuration \(application.properties\)](#)
11. [Glossary \(important terms & annotations\)](#)
12. [Interview presentation + Q&A bank](#)

1) One-page overview

AlertSphere is a Spring Boot backend where a user can sign up, sign in, create weather-alert rules for a city, and automatically receive an email when the weather matches their rule.

- **Auth:** Username/password login returns a JWT token. All protected endpoints require `Authorization: Bearer <jwt>`.
- **Persistence:** Users and Alerts are stored in MongoDB documents.
- **Automation:** A scheduler runs periodically to check weather for all active alerts.
- **Performance:** Redis caches weather responses per city (TTL).
- **Reliability:** Redis cooldown locks reduce duplicate notifications; Kafka decouples “detect” and “send email”.
- **Notification:** When an alert triggers → publish Kafka event → consumer sends email → DB marks alert triggered.

Important “real-world” note: production systems should not hardcode secrets in code/properties, and admin creation endpoints must be protected. This guide explains how things work and also highlights typical improvements to discuss in interviews.

2) Architecture & request lifecycle

2.1 Layered architecture

Think of your backend in layers (each layer has a job):

- **Controller layer** (API): HTTP in/out, validation, status codes.
- **Service layer** (Business): rule evaluation, workflows, orchestration.
- **Repository layer** (Data): MongoDB queries and persistence.
- **Integration layer**: Redis cache/locks, Kafka events, SMTP email, OpenWeather HTTP calls.

2.2 “What happens when a request arrives?”

For a protected endpoint like `GET /alert` :

1. **Spring Security filter chain** runs first.
2. **JwtFilter** tries to authenticate the request using the Bearer token.
3. If the token is valid, Spring sets **SecurityContext** with the authenticated user.
4. Spring checks authorization rules (e.g., “authenticated required”).
5. Controller method executes, reads the current user from SecurityContext, calls services, returns response.

2.3 “What happens on alert trigger?”

Background flow (no HTTP request):

1. Scheduler runs and loads active alerts.
2. For each city: get weather (Redis cache first, else OpenWeather API).
3. Evaluate alert rules in **AlertService.shouldTrigger**.
4. If triggered: acquire cooldown lock in Redis to avoid duplicates.
5. Publish event to Kafka; consumer sends email and marks DB state.

3) API endpoints & how to test

Base URL is usually `http://localhost:8080` unless configured.

Endpoint	Auth	Purpose
<code>GET /auth/health-check</code>	No	Simple health check
<code>POST /auth/signup</code>	No	Create a user
<code>POST /auth/signin</code>	No	Login and receive a JWT
<code>POST /alert/create-alert</code>	JWT	Create an alert rule
<code>GET /alert</code>	JWT	List current user's alerts
<code>GET /alert/{id}</code>	JWT	Fetch one alert
<code>PUT /alert/{id}</code>	JWT	Update an alert
<code>DELETE /alert/{id}</code>	JWT	Delete an alert
<code>POST /user/update-user</code>	JWT	Update username/password
<code>GET /admin/all-users</code>	ADMIN	Admin-only list users

3.1 Example test sequence (curl)

Note: In PowerShell, `curl` can be an alias of `Invoke-WebRequest`. To use real curl, prefer `curl.exe`.

1) Signup (PowerShell + curl.exe):

```
curl.exe -X POST "http://localhost:8080/auth/signup" -H "Content-Type: application/json" -d "{ \"userName\": \"alice\", \"password\": \"pass123\", \"email\": \"alice@example.com\" }\""
```

2) Signin (get JWT):

```
curl.exe -X POST "http://localhost:8080/auth/signin" -H "Content-Type: application/json" -d "{ \"userName\": \"alice\", \"password\": \"pass123\" }\""
```

3) Use JWT for protected API:

```
curl.exe "http://localhost:8080/alert" -H "Authorization: Bearer "
```

```
<JWT_FROM_SIGNIN>"
```

4) Create an alert rule:

```
curl.exe -X POST "http://localhost:8080/alert/create-alert" -H "Authorization: Bearer <JWT_FROM_SIGNIN>" -H "Content-Type: application/json" -d "{ \"city\": \"Delhi\", \"temperatureAbove\": 30, \"humidityBelow\": 80, \"conditionType\": \"AND\", \"cooldownMinutes\": 30, \"active\": true }"
```

Optional: The same using PowerShell `Invoke-RestMethod` :

```
$jwt = "<JWT_FROM_SIGNIN>"
$headers = @{ Authorization = "Bearer $jwt" }

Invoke-RestMethod -Method Get -Uri "http://localhost:8080/alert" -Headers $headers

$body = @{
    city = "Delhi"
    temperatureAbove = 30
    humidityBelow = 80
    conditionType = "AND"
    cooldownMinutes = 30
    active = $true
} | ConvertTo-Json

Invoke-RestMethod -Method Post -Uri "http://localhost:8080/alert/create-alert" -Headers $headers -ContentType "application/json" -Body $body
```

4) Spring Security + JWT (deep explanation)

How to study this section: First learn the vocabulary (Authentication/Authorization, Filter, SecurityContext). Then follow the two end-to-end flows: `/auth/signin` (login) and `/alert` (request with JWT). If you can explain those two flows clearly, you can explain the whole security system in an interview.

4.1 Authentication vs Authorization

- **Authentication** = prove who you are.
- **Authorization** = what you're allowed to do (roles/permissions).

4.2 Your security rules (who can call what)

In `SpringSecurity.securityFilterChain` you define rules:

- `/auth/**` → permit all (public endpoints)
- `/alert/**` and `/user/**` → must be authenticated
- `/admin/**` → must have role ADMIN

Code reference: `src/main/java/com/ishaan/AlertSphere/config/SpringSecurity.java:37`

4.3 Why stateless sessions?

`SessionCreationPolicy.STATELESS` means the server does not store login state in an HTTP session. Instead, the client sends a JWT on every request.

Why this is good: easier scaling (multiple server instances) + works well for REST APIs.

4.4 What is a “filter chain”?

A filter chain is a sequence of components that run before your controller method. Spring Security uses filters to implement authentication/authorization consistently for every request.

4.5 Why add `JwtFilter` before `UsernamePasswordAuthenticationFilter`?

You call `http.addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class)`.

The “why”:

- Authorization decisions are based on **SecurityContext** (who is authenticated).
- Your **JwtFilter** is what builds authentication from the Bearer token and stores it into `SecurityContext`.
- So the JWT filter must run early enough that, when Spring checks `.authenticated()` / `.hasRole()`, it can see the user.
- Using `UsernamePasswordAuthenticationFilter` as the anchor point is a common pattern (it's an early, known filter in the chain).

4.6 What is AuthenticationManager? Why do you use it in /signin?

AuthenticationManager is the component that performs login authentication. In your code, you call it during `POST /auth/signin`.

How it works (your exact flow):

1. User sends username & password to `/auth/signin`.
2. You create an authentication request object: `new UsernamePasswordAuthenticationToken(username, password)`.
3. You call `authenticationManager.authenticate(...)`.
4. Spring delegates to an **AuthenticationProvider**. You configured: `DaoAuthenticationProvider`.
5. `DaoAuthenticationProvider` calls your **UserDetailsService** to load user from MongoDB.
6. It uses **PasswordEncoder** (BCrypt) to compare raw password vs stored hash.
7. If valid → authentication success → you generate JWT and return it to the client.

Code references: `PublicController.java:59`, `SpringSecurity.java:51`, `UserDetailsServiceImpl.java:20`

4.7 What is DaoAuthenticationProvider? (meaning + why)

`DaoAuthenticationProvider` is a built-in provider that authenticates a user by loading their details via `UserDetailsService` and verifying the password with a `PasswordEncoder`.

Why we use it: it's the standard, secure way to implement username/password login in Spring Security.

4.8 What is UserDetailsService? Why you implement it?

Spring Security does not want to know your database details. It only needs a standard method: `loadUserByUsername`.

Your implementation loads `User` from MongoDB and returns Spring's `UserDetails` with roles.

Code reference: `UserDetailsServiceImpl.java:20`

4.9 Roles vs Authorities (interview favorite)

In your code you set roles like `USER`, `ADMIN`. Spring internally turns them into authorities like `ROLE_USER`, `ROLE_ADMIN`.

That's why `hasRole("ADMIN")` matches users whose authorities include `ROLE_ADMIN`.

4.10 JWT: what it is, what your code checks

JWT is a signed token. Your server signs it with a secret key; later you verify the signature to ensure the token wasn't modified.

- **Generate:** `JwtUtil.generateToken(username)`

- **Parse/verify signature:** `JwtUtil.extractAllClaims`
- **Validate:** in your code, validation checks expiration (signature is verified during parsing)

Code reference: `JwtUtil.java:30`, `JwtUtil.java:59`

4.11 JwtFilter: step-by-step for every request

For a request like `GET /alert` :

1. Read `Authorization` header.
2. If it starts with `Bearer`, extract the JWT string.
3. Extract username from token.
4. Load user details from DB.
5. If token not expired, build an authenticated token and set it into `SecurityContext`.
6. Continue to controller.

Code reference: `JwtFilter.java:27`

4.12 Deep vocabulary (the words interviewers expect)

- **Authentication:** An object that represents "someone proved who they are". In Spring it holds: principal (who), credentials (proof), and authorities (roles/permissions).
- **Principal:** Typically the username (or a full user object) representing the logged-in identity. In your code it is derived from the JWT's `sub` claim.
- **Authorities:** A list of permissions, like `ROLE_ADMIN`. Spring uses these for `hasRole` / `hasAuthority` checks.
- **SecurityContext:** The per-request "box" that stores the current Authentication. Controllers read it via `SecurityContextHolder`.
- **OncePerRequestFilter:** A filter base class that guarantees your filter runs once per request. Useful for JWT parsing because it should happen exactly once.

4.13 End-to-end flow #1: Login (/auth/signin)

This is how login works from HTTP request to JWT response:

1. Client sends username/password.
2. Controller creates a `UsernamePasswordAuthenticationToken` containing the raw credentials. This token is an *authentication request*.
3. `AuthenticationManager.authenticate` receives the request and delegates to a provider.
4. `DaoAuthenticationProvider` calls your `UserDetailsServiceImpl.loadUserByUsername`.
5. Your service loads the user from MongoDB and returns `UserDetails` including the stored BCrypt password hash and roles.

6. Provider uses your BCrypt `PasswordEncoder` to compare the raw password and the stored hash. If it matches → authentication success.
7. Controller generates JWT and returns it.

4.14 End-to-end flow #2: Protected request with JWT (GET /alert)

This is how a JWT-secured endpoint works:

1. Client calls `GET /alert` with header `Authorization: Bearer <jwt>`.
2. `JwtFilter` runs before controllers.
3. It parses the token, verifies signature (during claim parsing), checks expiration, loads user details.
4. It creates an authenticated `UsernamePasswordAuthenticationToken` and stores it in `SecurityContextHolder`.
5. Spring evaluates authorization rules; since user is authenticated, it allows access.
6. Controller reads the username from `SecurityContext` and loads the user record to filter alerts by `userId`.

4.15 Why do we load userDetails during JWT validation?

In your implementation, after extracting username, you call `userDetailsService.loadUserByUsername`.

Reason:

- To attach correct authorities/roles to the Authentication object.
- To ensure the user still exists (deleted users should not be authenticated).
- In more advanced systems: to compare token against a “token version” or revoked tokens list.

5) Business layer (services) breakdown

The **service layer** is your “use-case layer”. It contains the actual business rules and orchestration.

5.1 UserService — why it exists

File: `src/main/java/com/ishaan/AlertSphere/service/UserService.java`

Purpose: all user creation rules belong here, not in controllers.

- Hashes passwords with BCrypt (never store plain password).
- Assigns roles.
- Saves user document to MongoDB.

5.2 AlertService — the core domain logic

File: `src/main/java/com/ishaan/AlertSphere/service/AlertService.java`

5.2.1 Create alert

`createAlert(request, user)` builds a new `Alert` document:

- Copies rule fields from request (temperature/humidity/wind/condition).
- Applies defaults (active, condition type, cooldown).
- Stores `userId` and `userEmail` into the alert (so scheduler can email).
- Saves into MongoDB repository.

5.2.2 Trigger evaluation algorithm (shouldTrigger)

`shouldTrigger(alert, weather)` decides if an alert matches current weather.

Step 1: guard checks

- If not active → do not trigger.
- If in cooldown (`lastTriggeredAt` + `cooldownMinutes` not passed) → do not trigger.

Step 2: build condition checks

The service creates a list of boolean checks only for fields the user configured:

`tempAbove`, `tempBelow`, `humidityAbove`, etc.

Step 3: apply AND/OR rule

- If condition type is OR → any check true triggers.
- Else AND → all checks must be true.

Interview-level explanation: this pattern is a small “rule engine”. You convert a user-defined rule into independent checks and then fold them using AND/OR.

5.3 WeatherAlertScheduler — orchestration

The scheduler is what turns your app into an automated system, not just CRUD APIs.

- Runs on a schedule (fixed delay from properties).
- Loads all active alerts.
- Groups by city to avoid duplicate weather calls.
- For each city, fetch weather, find triggered alerts, lock cooldown, publish email events.

6) Data layer (MongoDB) breakdown

6.1 What is a “Document”?

MongoDB stores JSON-like records called **documents**. A collection is like a table.

- `User` is stored in collection `user`.
- `Alert` is stored in collection `alerts`.

6.2 Why indexes matter

- `@Indexed(unique=true)` on username enforces uniqueness and speeds lookups.
- Indexing city and userId speeds “find alerts by city/user” queries used by scheduler and APIs.

6.3 Repositories

Spring Data creates implementations automatically for methods like: `findById`, `findByActiveTrue`, `findByCityIgnoreCaseAndActiveTrue`.

7) Redis caching + cooldown locks

7.1 Weather cache (performance)

Weather responses are cached in Redis by city for a TTL (minutes). This reduces OpenWeather calls and speeds up scheduler runs.

Why TTL? weather changes; TTL balances freshness and performance.

7.2 Cooldown locks (anti-duplication)

Even if your business cooldown exists in MongoDB, multiple app instances or rapid scheduler runs can still cause duplicates. Redis `SETNX + TTL` acts as a distributed lock:

- If the lock key is absent, set it and proceed to send.
- If key already exists, skip sending (someone already handled it recently).

Why return true when Redis is down? fail-open avoids missing alerts; you accept possible duplicates rather than losing notifications.

8) Kafka event pipeline + reliability

8.1 Why Kafka here?

Kafka decouples “detect triggered alert” from “send email”. This allows scaling email workers and prevents the scheduler from being blocked by slow SMTP.

8.2 Producer send is asynchronous

`kafkaTemplate.send(...)` returns a future; your code registers `whenComplete` to log success/failure. Failures fallback to direct send.

8.3 Consumer groups and offsets (conceptual)

- **Topic** = stream of messages.
- **Consumer group** = many consumers share work; each message goes to exactly one consumer in the group.
- **Offset** = Kafka’s pointer for “how far you’ve read”.
- **At-least-once delivery** is common; duplicates can happen. Your cooldown lock helps reduce duplicates.

9) Email delivery (SMTP)

Your app uses Spring Mail (`JavaMailSender`) to send an email with subject/body. SMTP server details come from properties.

- From: configured mail username
- To: user's email from the alert event
- Body: formatted alert details (city/temp/humidity/wind/condition/time)

10) Configuration (application.properties)

Spring Boot reads configuration from `src/main/resources/application.properties`. These values configure database connections, Redis, Kafka, mail, and the scheduler.

Best practice: do not commit secrets. Use environment variables or a secret manager. For learning, it's OK, but in interviews mention you would externalize secrets.

11) Glossary (key terms & “what each word means”)

Spring / Java words

- **Bean:** an object created and managed by Spring.
- **Dependency Injection:** Spring provides required objects automatically.
- **IoC Container:** Spring’s runtime that stores and wires beans.
- **DTO:** “Data Transfer Object” used for requests/responses.
- **Entity/Document:** object stored in the database.
- **Repository:** abstraction for database operations.

Security words

- **Principal:** the “identity” (username/user).
- **Credentials:** proof (password/token).
- **Authority:** permission string (e.g., ROLE_ADMIN).
- **Authentication:** verified identity.
- **Authorization:** permission to access a resource.
- **Stateless:** server does not store session; client sends token each time.

Infra words

- **MongoDB:** document database.
- **Redis:** in-memory key/value store; great for cache and locks.
- **Kafka:** distributed log / messaging platform.
- **SMTP:** protocol to send email.
- **TTL:** time-to-live (auto-expire data after time).

12) Interview presentation + Q&A bank

12.1 2–3 minute presentation script

AlertSphere is a Spring Boot backend for automated weather notifications. Users sign up and authenticate using JWT. Alerts are stored in MongoDB as documents with rule fields such as temperatureAbove, humidityBelow, and windSpeedAbove, combined using AND/OR logic. A scheduled job periodically loads active alerts, groups them by city, and fetches current weather. To reduce external API calls, weather is cached in Redis with TTL. When an alert triggers, a Redis cooldown lock prevents duplicate notifications. The app publishes a weather-alert event to Kafka, and a Kafka consumer sends the email using SMTP and then marks the alert as triggered in MongoDB. If Kafka is unavailable, the producer falls back to direct email delivery. This design is stateless, scalable, and demonstrates layered architecture and event-driven communication.

12.2 Common interviewer questions (with strong answers)

1. Why JWT instead of sessions?

Stateless APIs scale well. The server doesn't store session state; each request carries a signed token.

2. Why do you add JwtFilter before UsernamePasswordAuthenticationFilter?

So authentication is established early in the filter chain before authorization rules are evaluated.

3. What is AuthenticationManager?

The central component that performs authentication by delegating to providers like DaoAuthenticationProvider.

4. Why Redis when MongoDB already stores cooldown?

Redis provides fast distributed locking to prevent duplicates under concurrency or multiple instances.

5. Why Kafka?

It decouples detection from delivery, improves scalability, and isolates slow email sending from the scheduler.

6. What happens if Redis/Kafka is down?

The app degrades gracefully: weather cache falls back to API; event publishing falls back to direct send.

7. What improvements would you make for production?

Externalize secrets, protect admin creation, add structured error responses, add API documentation (OpenAPI), add observability and more tests.

5.2.3 Why store userEmail on Alert?

Your Alert document stores both `userId` and `userEmail`. This reduces lookups during email sending: the event already has the email address.

Tradeoff: email changes require keeping alerts updated (or you accept “email at creation time”).

5.2.4 Cooldown logic (MongoDB layer)

Cooldown prevents sending too frequently. In MongoDB you keep `lastTriggeredAt` and `cooldownMinutes`.

When an alert triggers, you update `lastTriggeredAt`. Next evaluation checks: `lastTriggeredAt + cooldownMinutes` must be before “now”.

5.4 WeatherService + WeatherCacheService

WeatherService calls the external API and builds `WeatherSnapshot`.

WeatherCacheService wraps WeatherService with Redis caching:

- Cache key normalization makes “Delhi” and “delhi” the same cache key.
- It catches Redis failures and continues (graceful degradation).
- TTL is configurable via properties.

5.5 AlertCooldownService — why a distributed lock

The Redis lock prevents duplicates when multiple scheduler executions overlap or multiple instances run the same job. It’s effectively a “try-lock” that expires automatically.

5.6 Kafka email pipeline services

- **AlertEventProducer:** publishes an event; if Kafka fails, sends directly.
- **AlertEmailConsumer:** reads events and triggers delivery.
- **AlertEmailDeliveryService:** sends and marks triggered in DB.
- **EmailNotificationService:** composes and sends SMTP email.

Annotation cheat-sheet (what each @ means)

- **@RestController:** class contains HTTP endpoints returning JSON/text.
- **@RequestMapping:** base path for all endpoints in a controller.
- **@GetMapping / @PostMapping / @PutMapping / @DeleteMapping:** maps HTTP method + path to a Java method.
- **@Service:** marks business-layer bean.

- **@Configuration**: class that defines beans and configuration.
- **@Bean**: method produces a bean instance for Spring to manage.
- **@EnableWebSecurity**: enables Spring Security integration.
- **@EnableScheduling**: enables scheduled jobs.
- **@Scheduled**: runs a method on a schedule.
- **@EnableKafka**: enables Kafka listener infrastructure.
- **@KafkaListener**: marks a method that consumes Kafka messages.
- **@ConditionalOnProperty**: bean is created only if a property matches; used for feature toggles.
- **@Valid**: triggers validation of request DTOs using Jakarta Validation annotations.
- **@Value**: injects a property value from configuration.